Team : NEVER PAUSE UNIT

NPU Challenge

NPU : Never Pause Unit 연구 진행 발표

박준범 윤수민 최민기 최윤성 | 2026.07.14

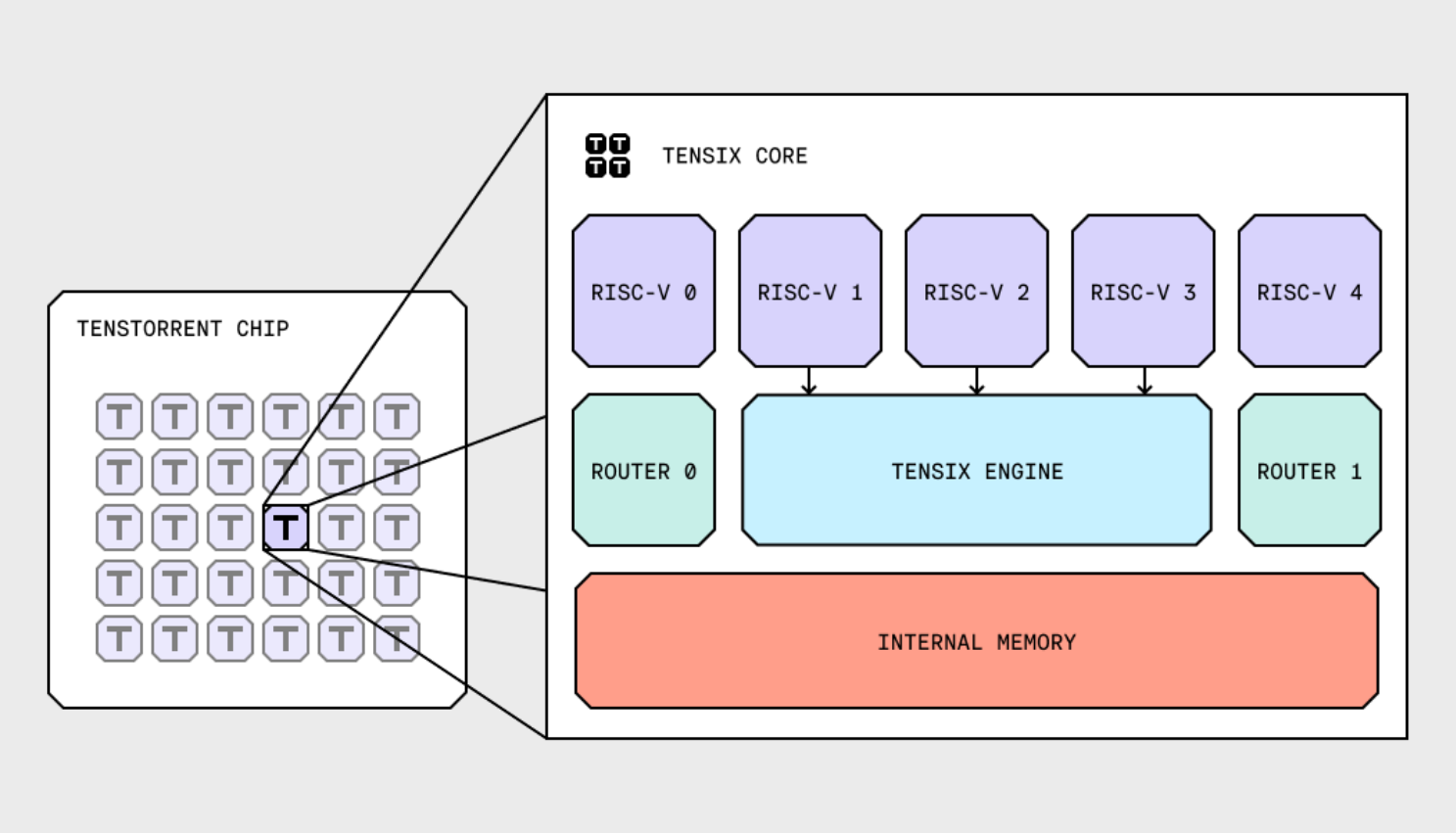
NPU Challenge Timeline

- Week 1 하드웨어 아키텍쳐 이해
- Week 2 소프트웨어 스택 이해 , AI model op 별 profiling
- Week 3 연구 주제 심화
- Week 4~9 연구 활동

Week 1
Tenstorrent Blackhole Architecture



Week1 : Tenstorrent Architecture 학습



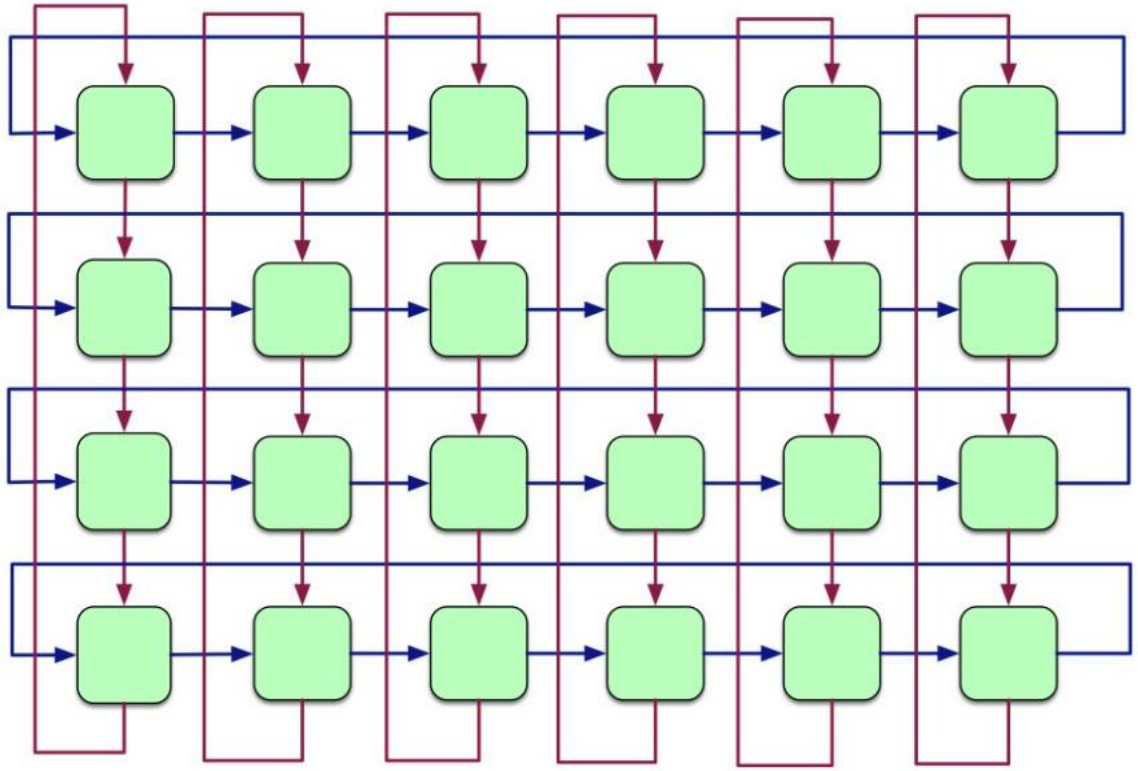
- Dual NoC
- L1 SRAM + CB
- Tensix Engine
- DRAM

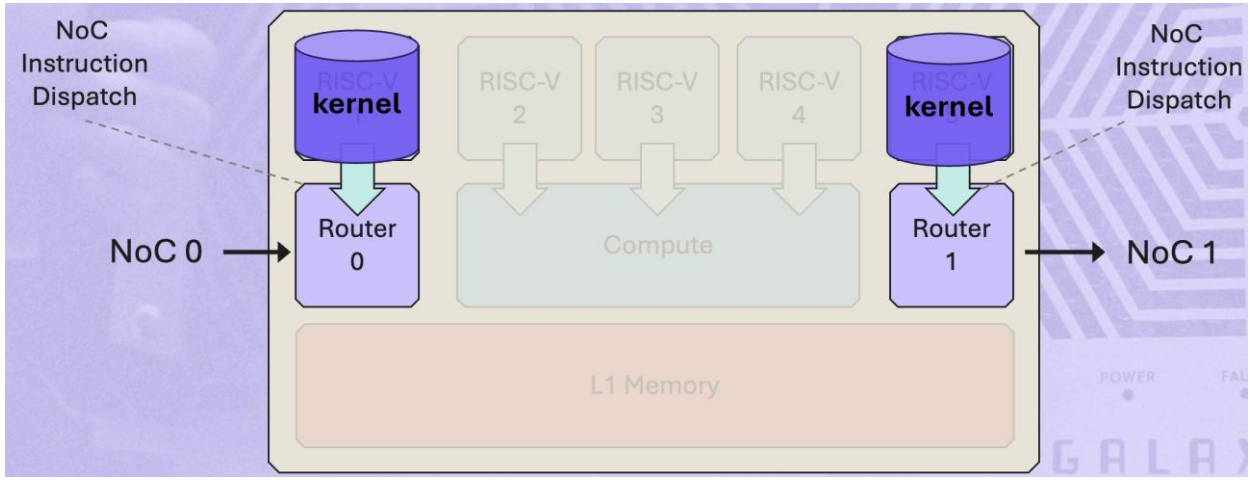
  -Blackhole : GPDDR6

  -Eagle-N : LPDDR5

https://github.com/tenstorrent/tt_metal/tt-llk/docs/llk/l2/images/tensix_engine.png

NoC (Network On Chip)

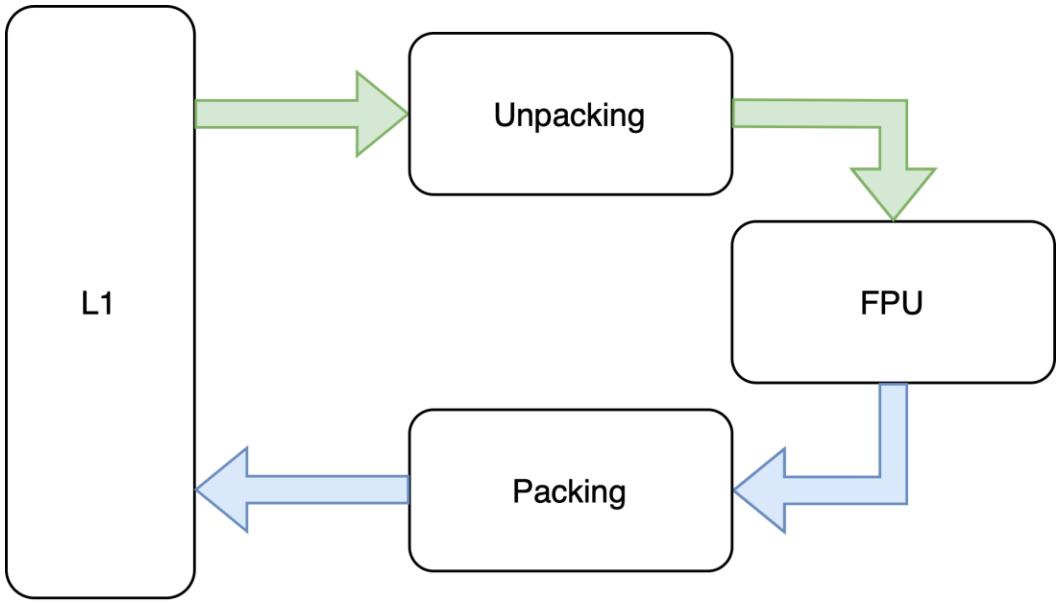


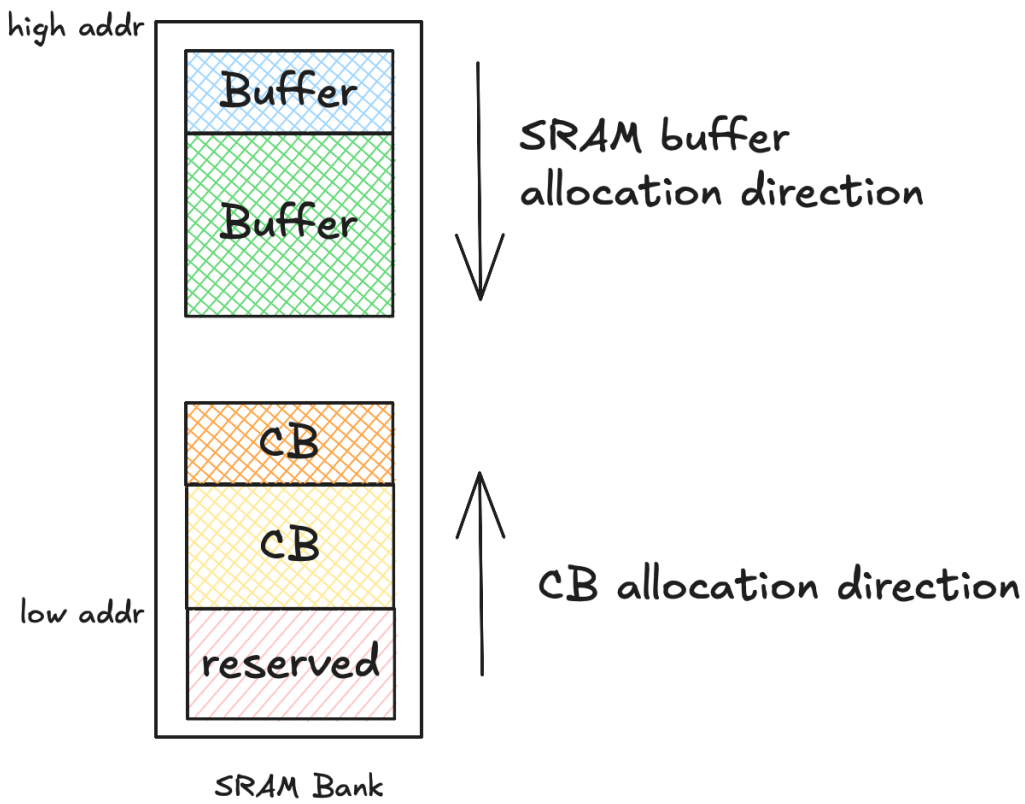


- Dual NoC (NOC_0 / NOC_1, XY routing)
- NoC congestion issue

https://github.com/tenstorrent/tt-metal/tree/main/docs/source/common/images

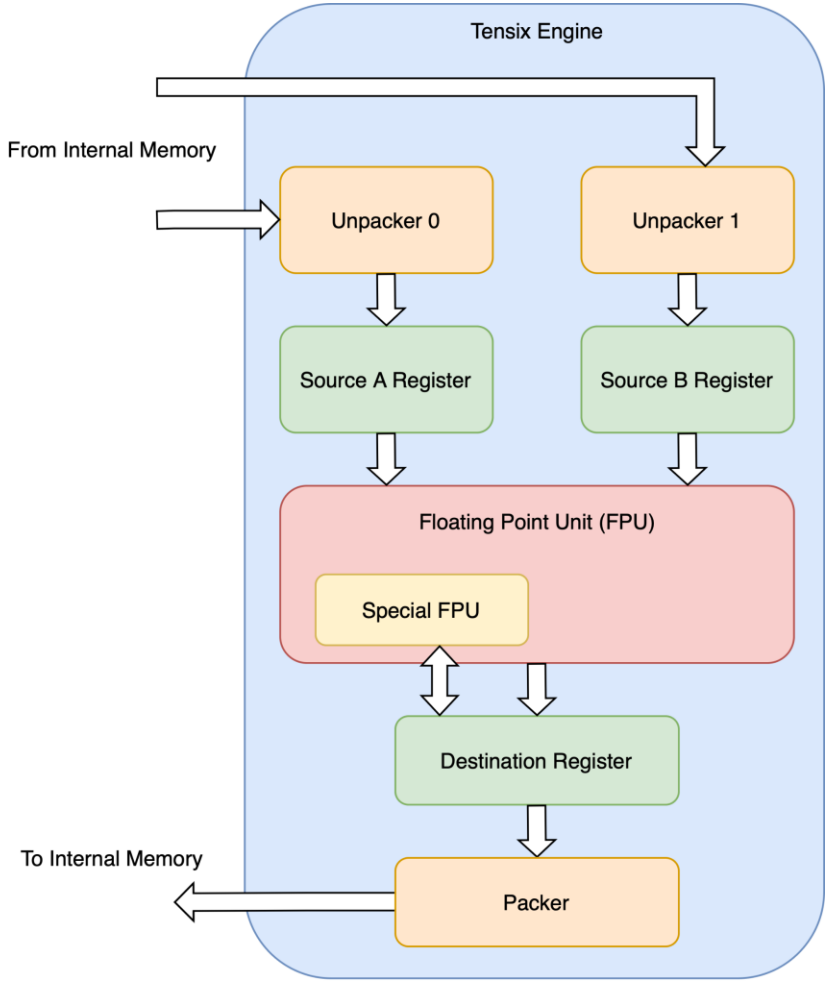
L1 SRAM per core + Circular Buffer

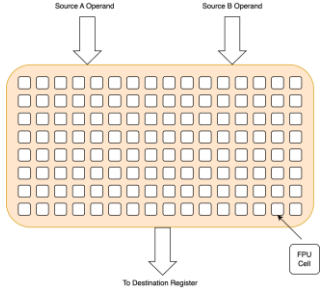


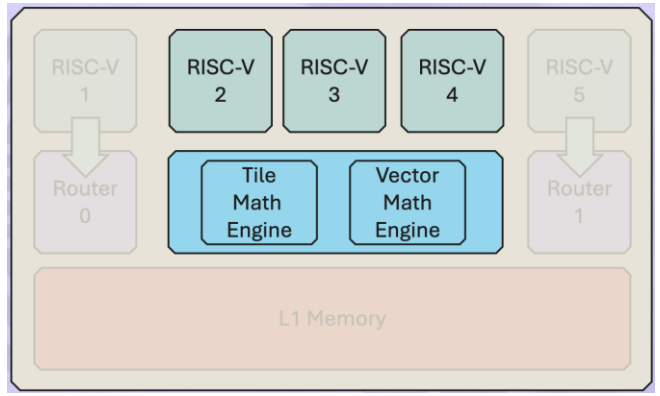


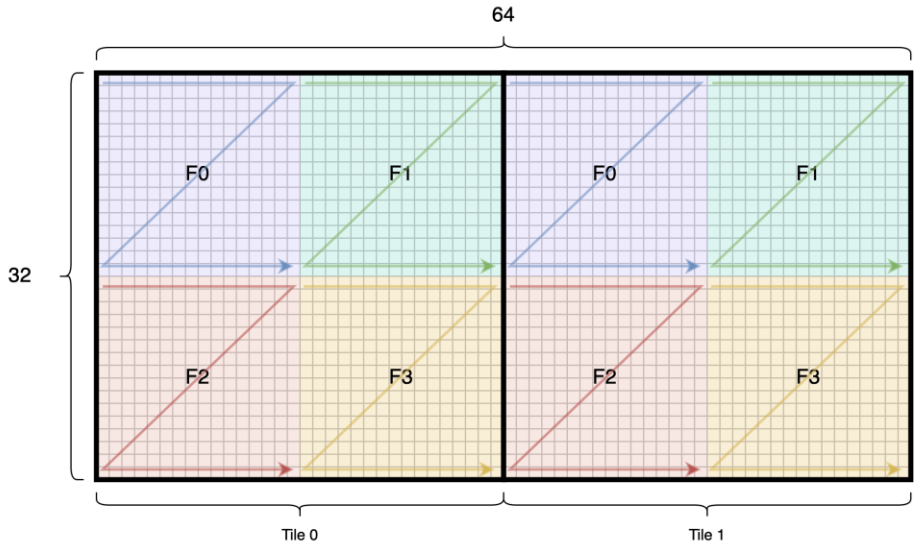
https://github.com/tenstorrent/tt-metal/blob/main/tech_reports/memory/allocator.md

Tensix Compute Engine





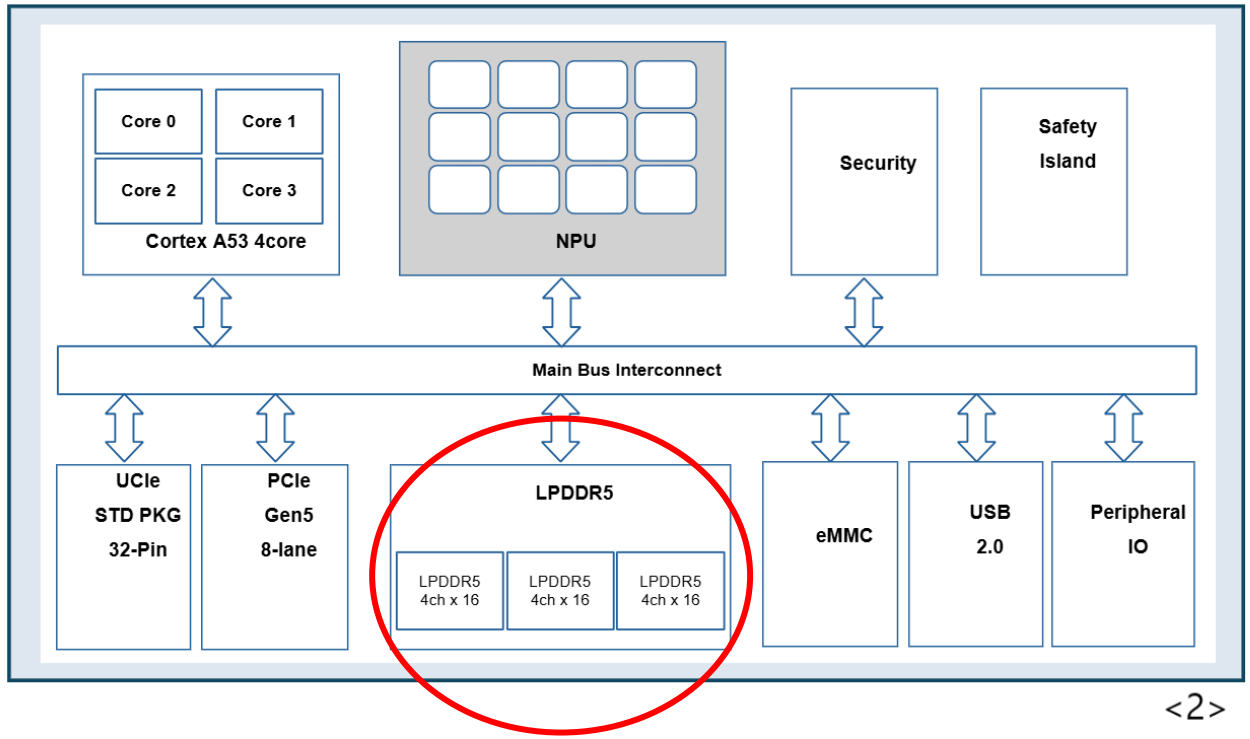




https://github.com/tenstorrent/tt-metal/blob/main/METALIUM_GUIDE.md

DRAM (LPDDR5, off-chip)

- SoC Block Diagram



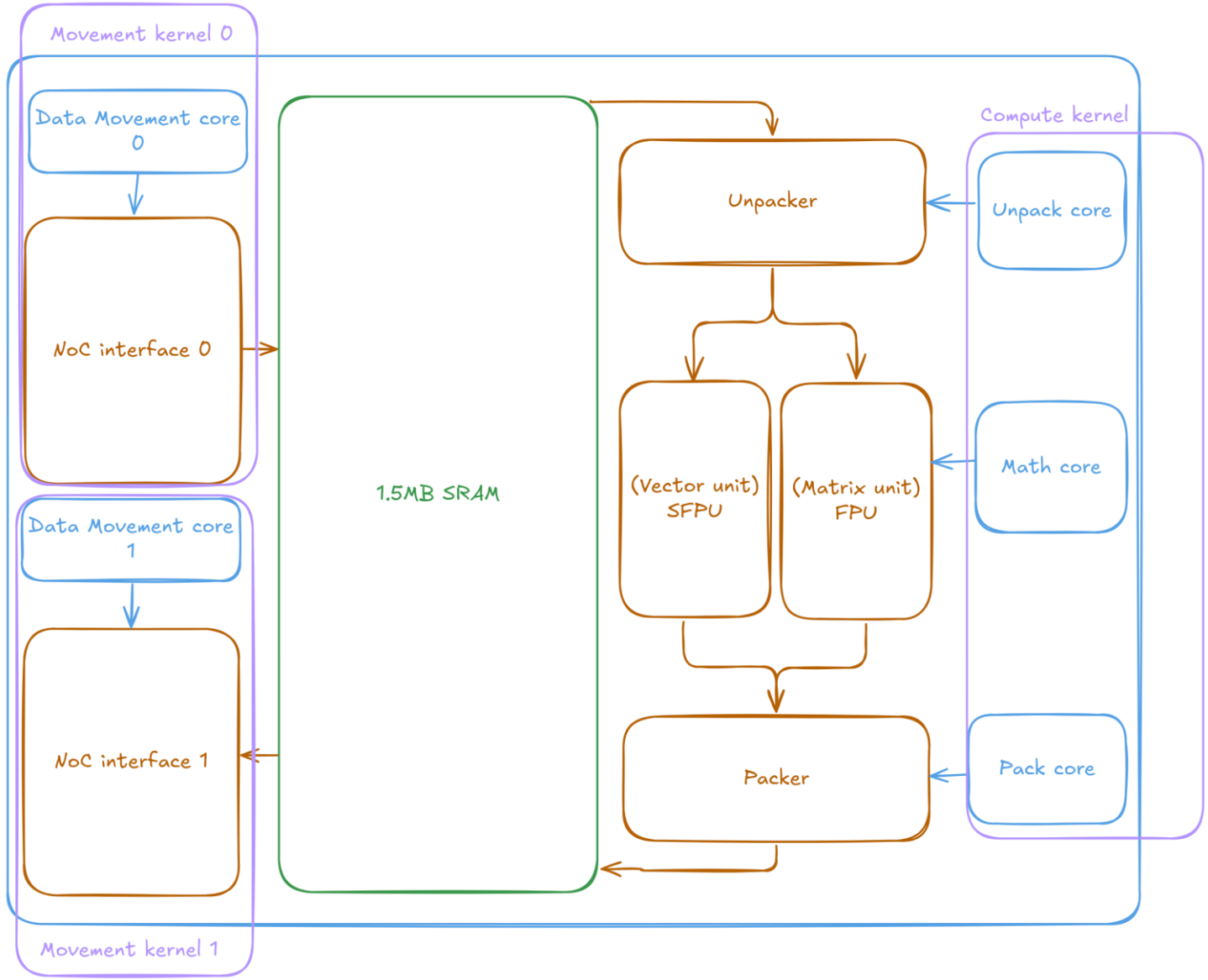
<2>

- DRAM — LPDDR5 (3 controller × 4 channel)

Week 2
AI 모델 & SW 스택

Tensix core Kernel

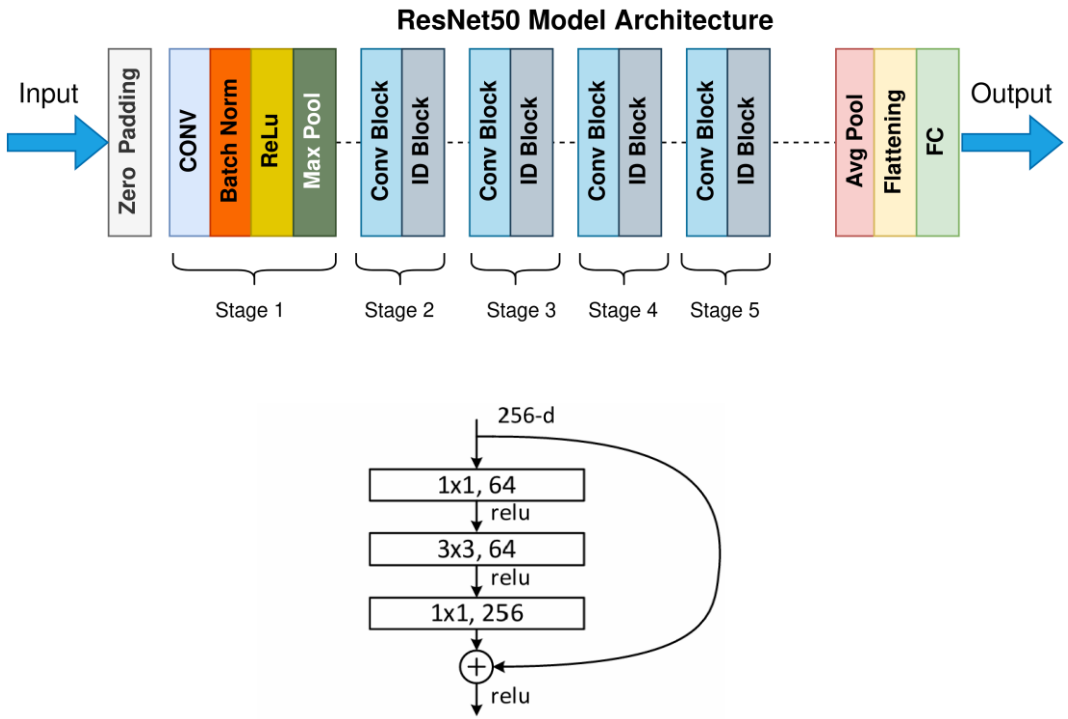
- Read Kernel
- Compute Kernel
- Wirte Kernel



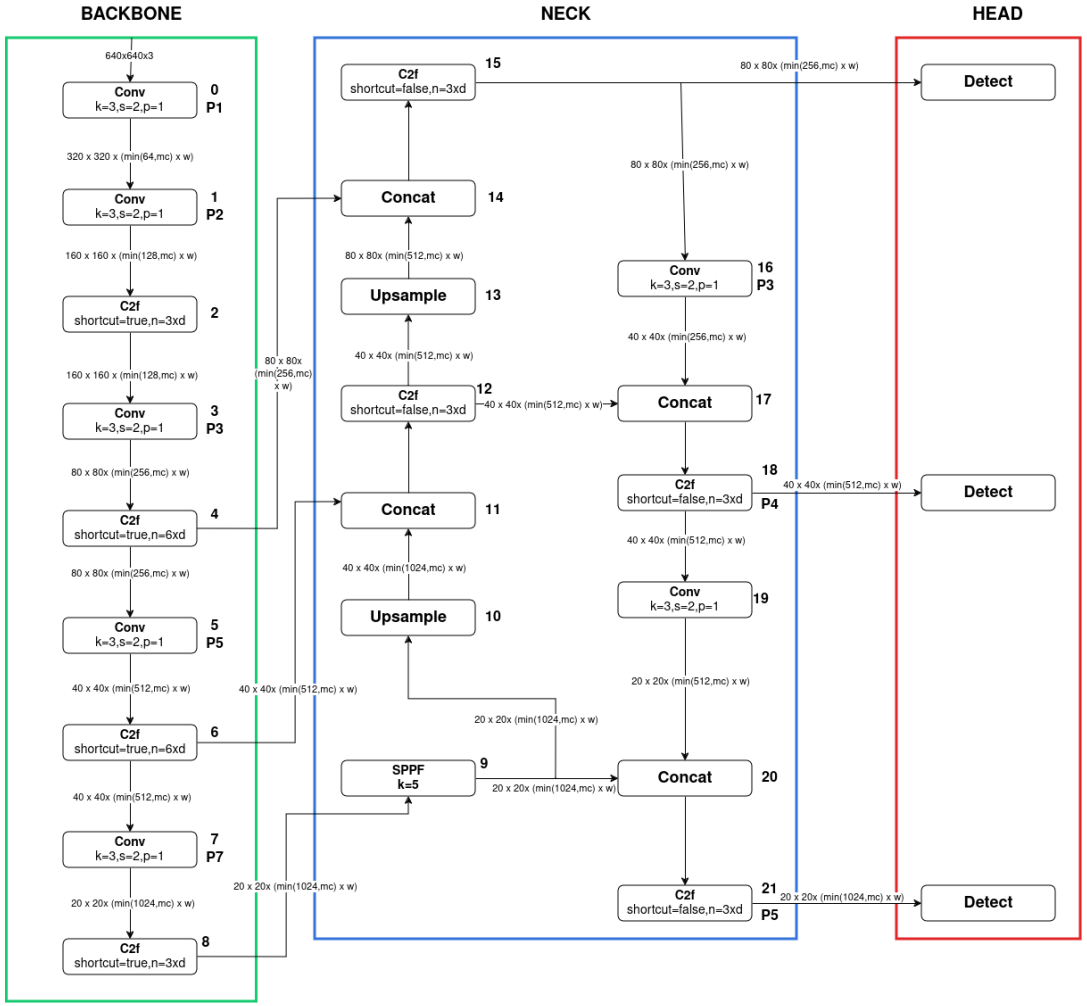
https://github.com/tenstorrent/tt-metal/blob/main/METALIUM_GUIDE.md

Baseline AI-model

ResNet50



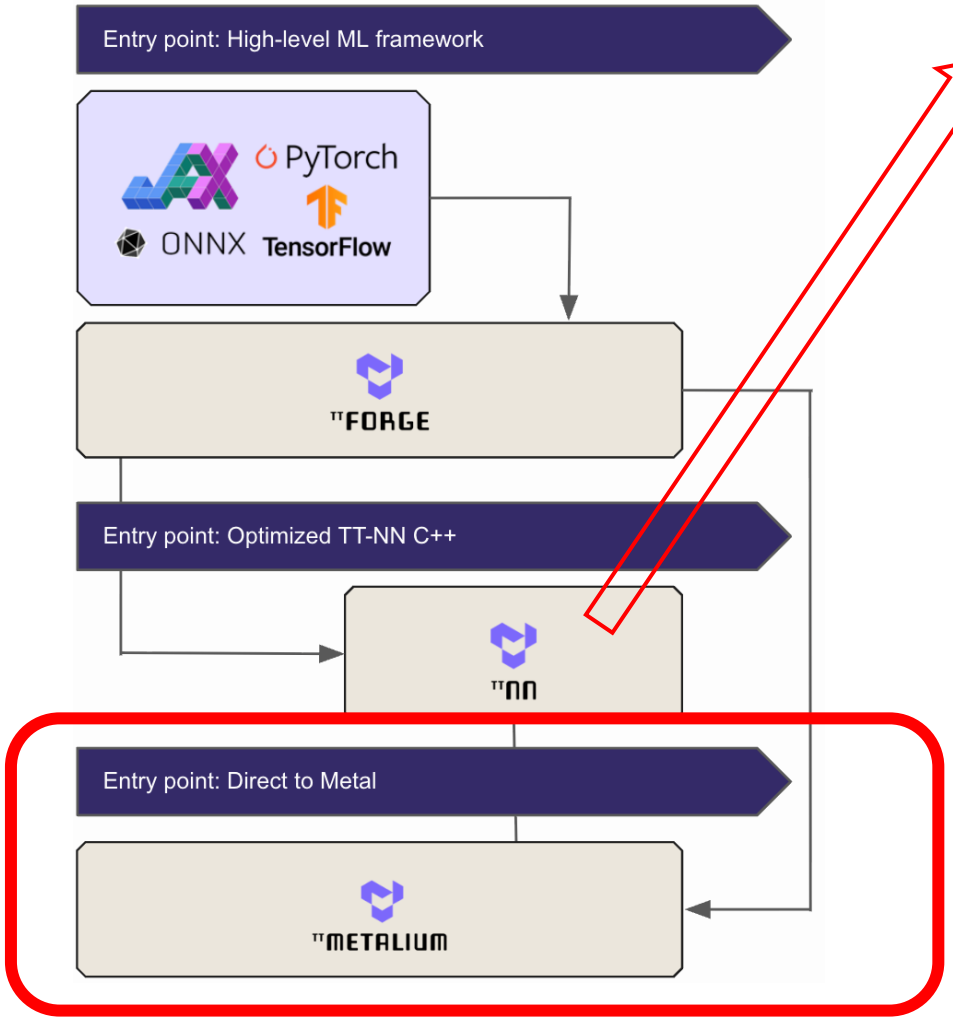
YoloV8s



He, K., Zhang, X., Ren, S., & Sun, J. (2016). "Deep Residual Learning for Image Recognition." CVPR 2016. arXiv:1512.03385 / Ultralytics YOLOv8 Documentation (2023)

소프트웨어 스택

스택 층별 접근 TT-NN → Metalium → LLK



TT-NN OP 호출

```
38      conv_config = ttnn.Conv2dConfig(
39          weights_dtype=weights_dtype,
40          # activation=None
41          deallocate_activation=deallocate_activation,
42          reallocate_halo_output=reallocate_halo_output,
43          # config_tensors_in_dram=False,
44          act_block_h_override=act_block_h_override,
45          # act_block_w_div=1,
46          reshard_if_not_optimal=reshard_if_not_optimal,
47          # override_sharding_config=False,
48          shard_layout=map_shard[shard_layout],
49          core_grid=core_grid,
50          transpose_shards=transpose_shards,
51          # output_layout=output_layout,
52          enable_act_double_buffer=False,
53          # enable_weights_double_buffer=False,
54          # full_inner_dim=False,
55          # in_place=False,
56          # enable_kernel_stride_folding=False,
57          # enable_activation_reuse=False,
58          # force_split_reader=None,
59      )
60
61      return conv_config
```

OP 동작

```
218     const bool overlap_im2col_cb =
219         sharding_scheme == TensorMemoryLayout::BLOCK_SHARDED && conv_input_df == output_df && !skip_act_cb_create;
220     {
221         // ACT and ACT_SECOND_READER CB
222         if (conv_config.enable_act_double_buffer) {
223             act_block_num_tiles *= 2;
224             act_block_split_num_tiles *= 2;
225         }
```

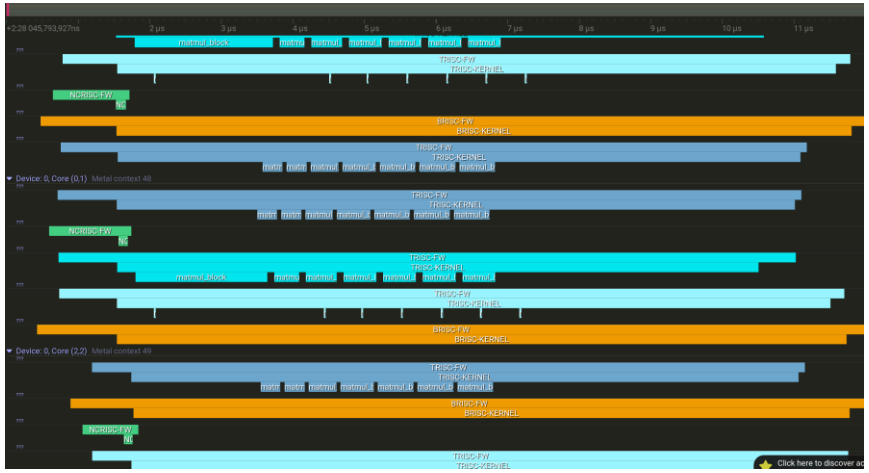
https://docs.tenstorrent.com/getting-started/tt-software-stack.html

Profiling Tool

TT-visualizer



Tracy



TT-npe

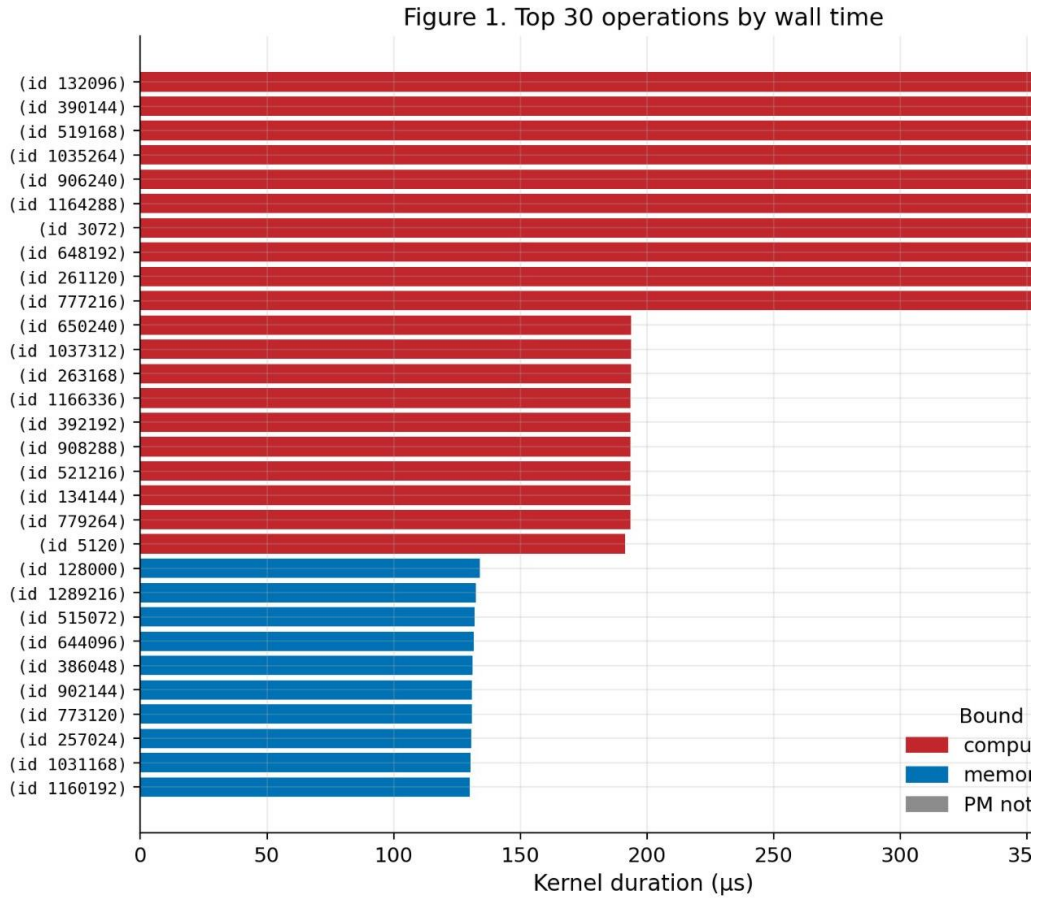


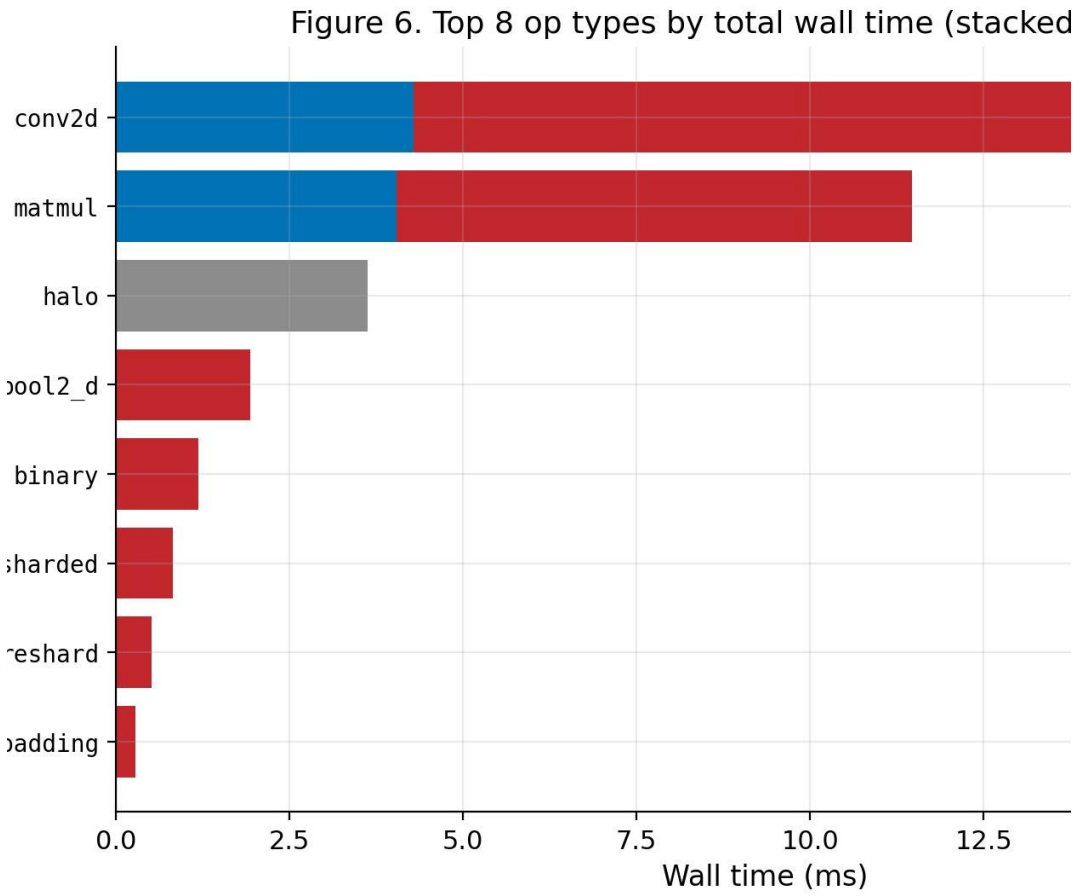
bos
SEMICONDUCTORS

Week3
Baseline 프로파일링 & 연구 방향 탐색

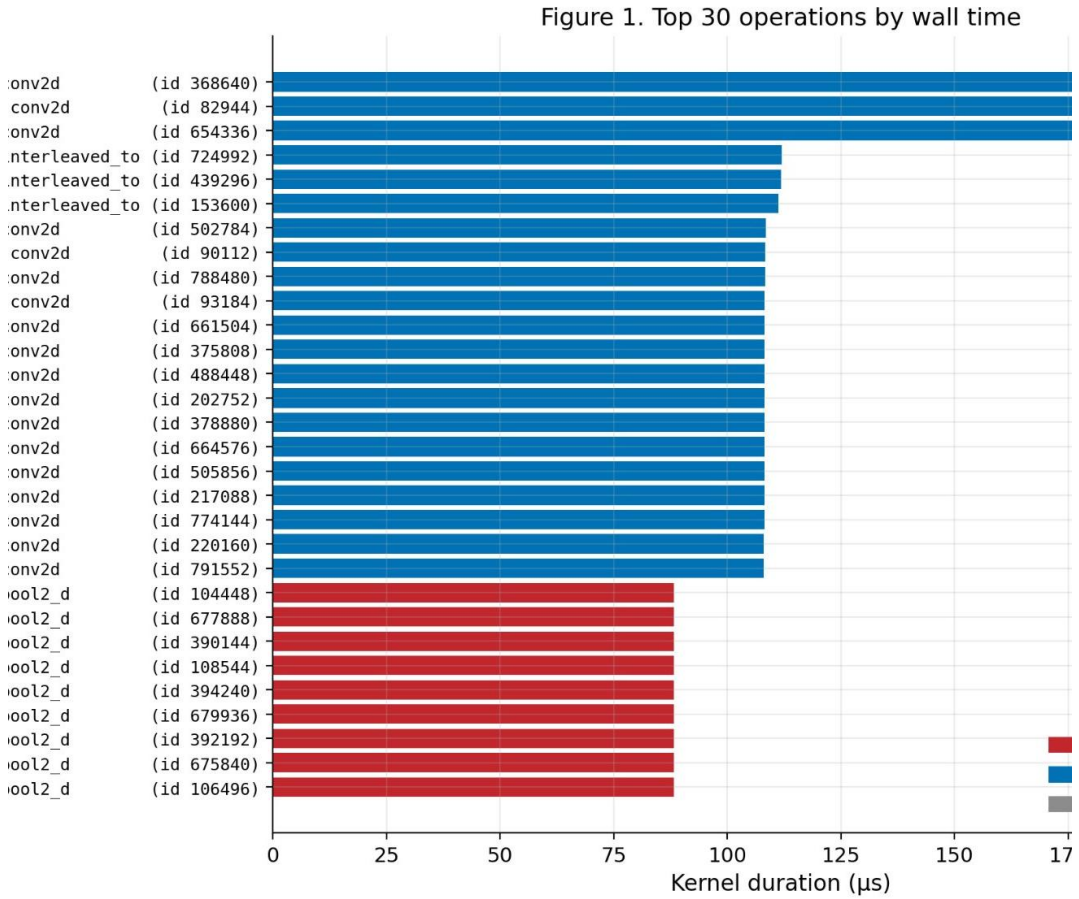
1. 성능 최적화
2. SubDevice를 활용한 여유 코어 활용

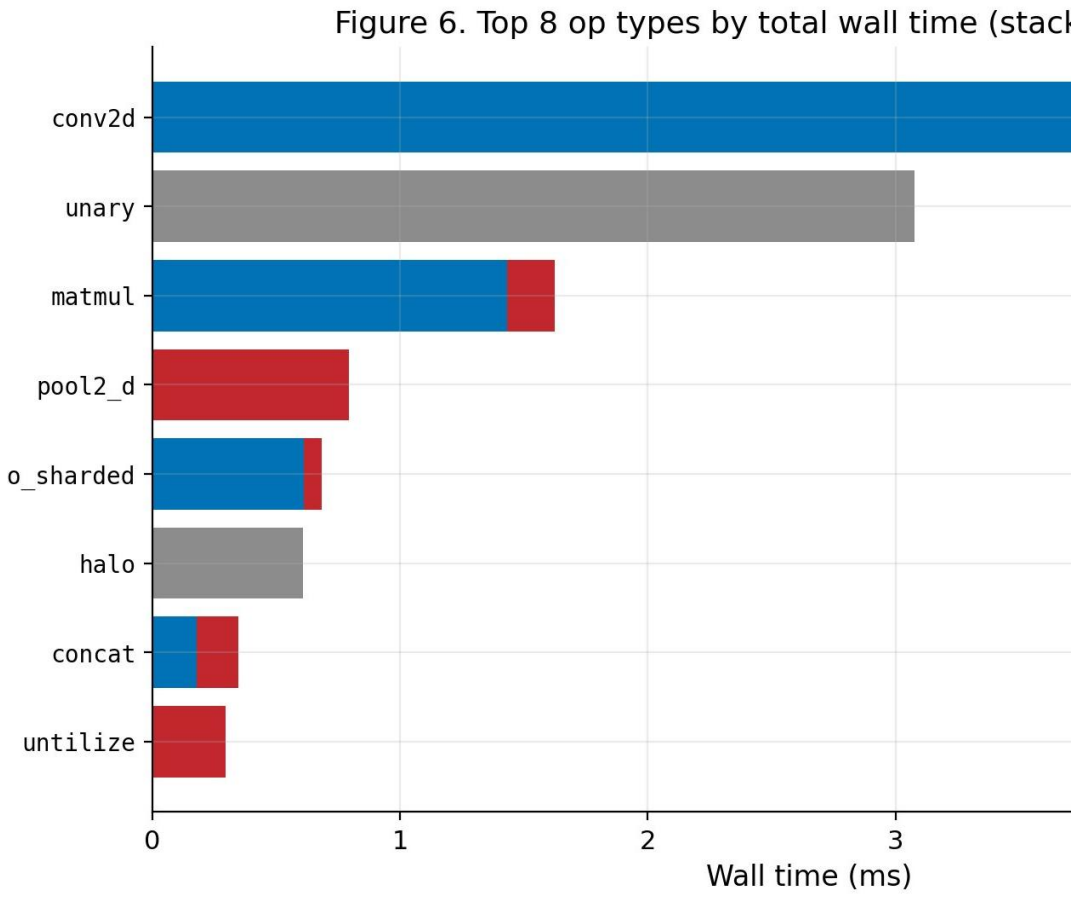
ResNet50 op별 wall time



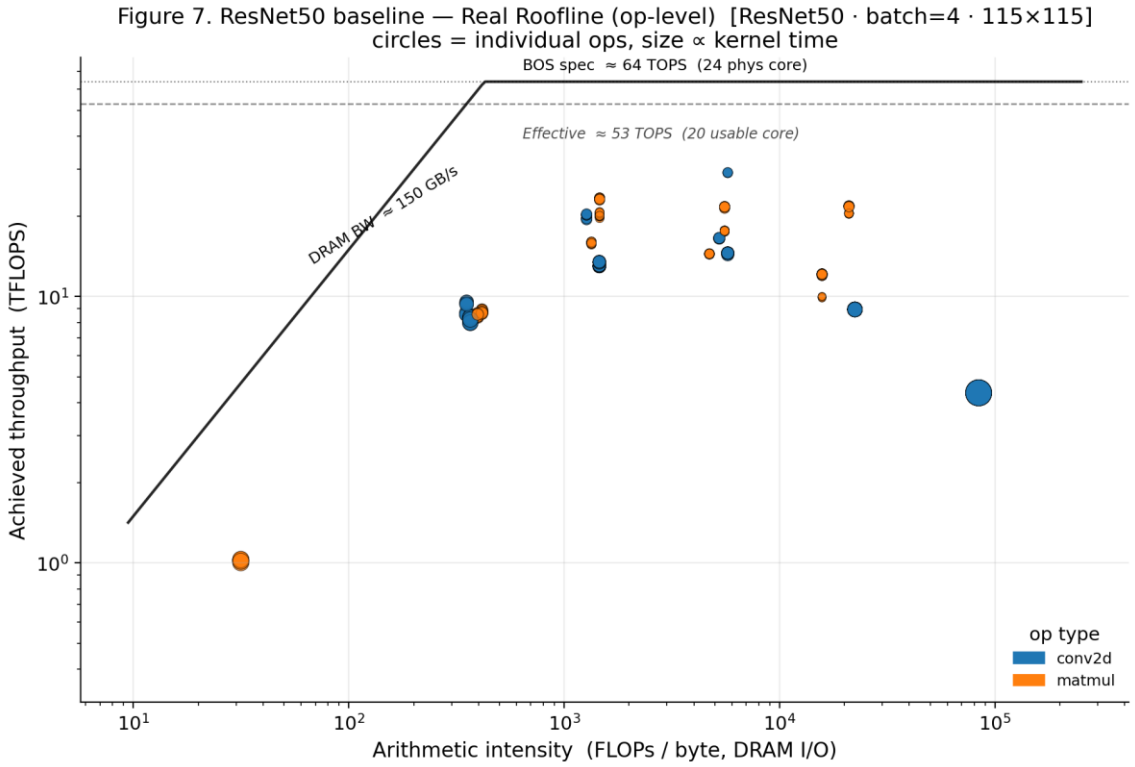


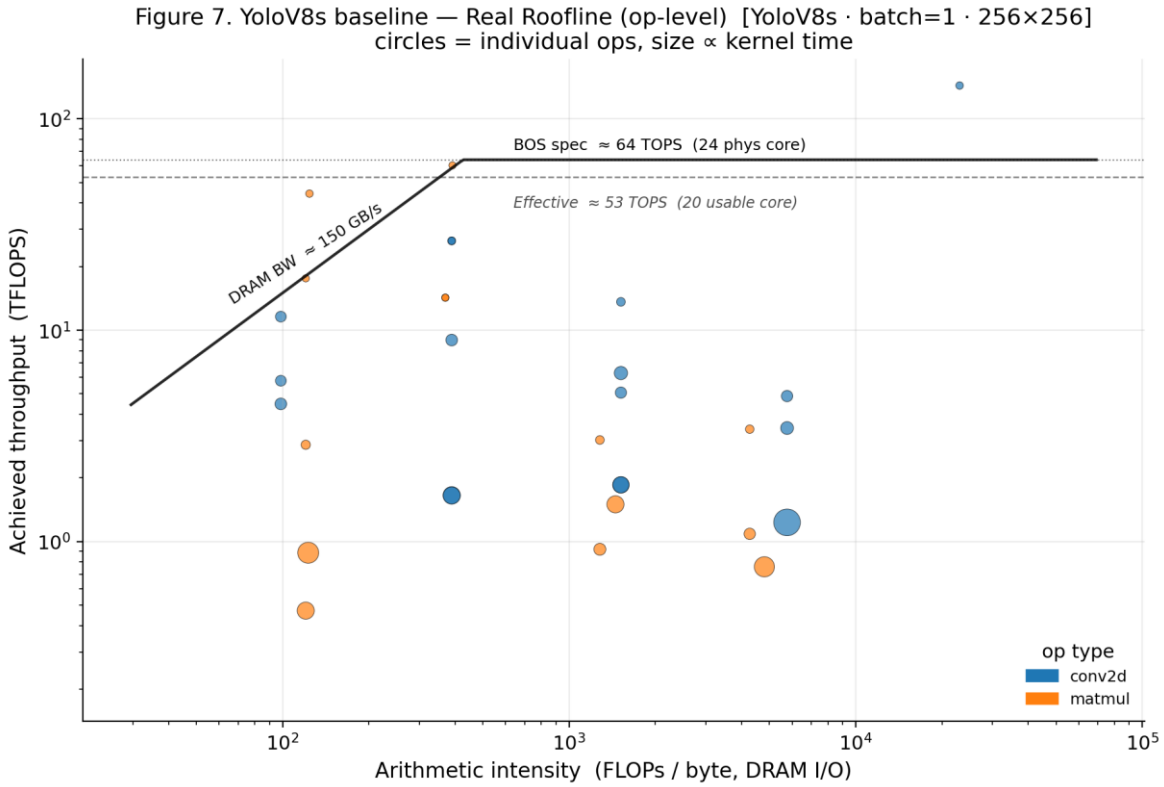
Yolo op별 wall time





Roofline Graph

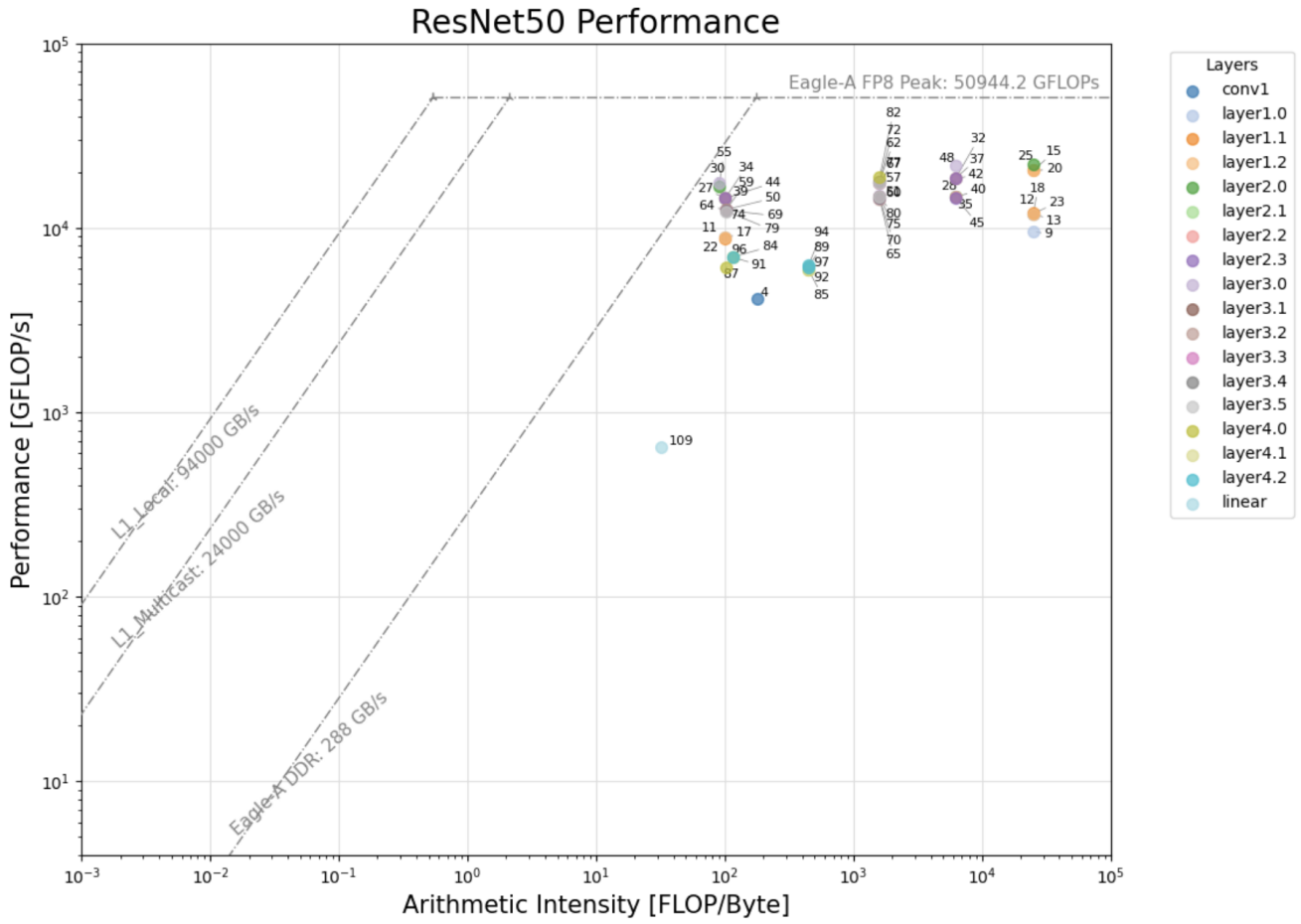




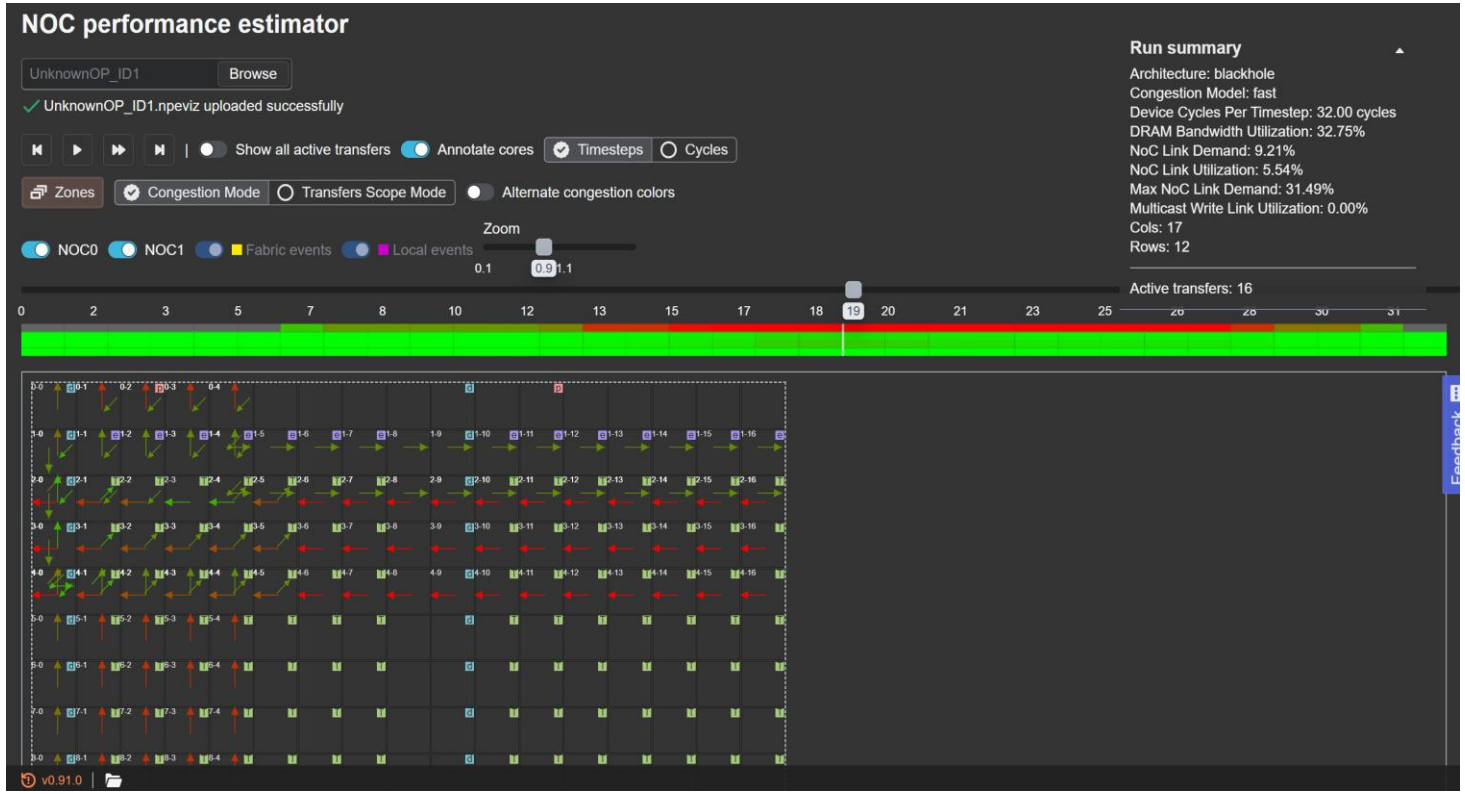
매트릭스 곱셈만 DRAM 접근 → activation 이 L1 에 잘 관리되고 있음
bytes_dram · peak값이 추정치. (NoC 트래픽 실측으로 검증 예정)

Roofline Graph 개선 방안

한계: spatial NPU L1 특성 X축 밖 → hierarchical 확장 필요



TT-npe 측정 작업 환경 구축



- TT-metal과 TT-npe 소프트웨어 conflict
- Blackhole 기준으로 시각화
- 한번에 돌릴 수 있는 op 수 제한.
  -> 코드를 split해서 돌릴 필요성 존재.

Error Message

```
394    TypeError: Stats.getDatapointByID() missing 2 required positional arguments: 'metal_trace_id' and
       'metal_trace_replay_session_id'
```

TT-npe 측정 작업 환경 구축

```
593         if STAGE == 1:
594             x_height, x_width = 56, 56
595             x = _synth_layer_input(device, self.batch_size, 56, 56, 64, self.model_config["ACTIVATIONS_DTYPE"])
596         if STAGE in (-1, 1):
597             # layer1
598             x, x_height, x_width = self._forward_layer(x, "layer1", self.batch_size, x_height, x_width, device)
599             if STAGE == 1:
600                 return x
601 
602         # ==== STAGE 2: layer2 ====
603         if STAGE == 2:
604             x_height, x_width = 56, 56
605             x = _synth_layer_input(device, self.batch_size, 56, 56, 256, self.model_config["ACTIVATIONS_DTYPE"])
606         if STAGE in (-1, 2):
607             # layer2
608             x, x_height, x_width = self._forward_layer(x, "layer2", self.batch_size, x_height, x_width, device)
609             if STAGE == 2:
610                 return x
611 
612         # ==== STAGE 3: layer3 ====
613         if STAGE == 3:
```

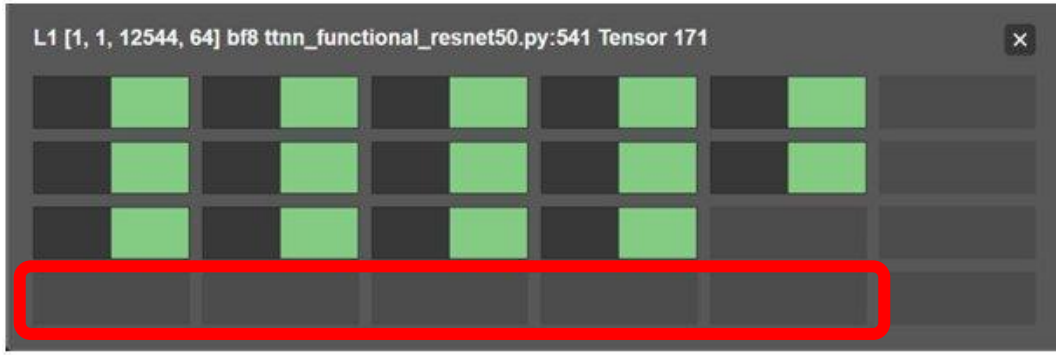
Sub-device Partition

compute_with_storage_grid_size() 확인 결과

`Compute grid size: (x=5, y=4)` → 즉 **5×4 = 20개 코어**가 ttnn이 실제로 계산/저장에 쓸 수 있는 전체 코어 수

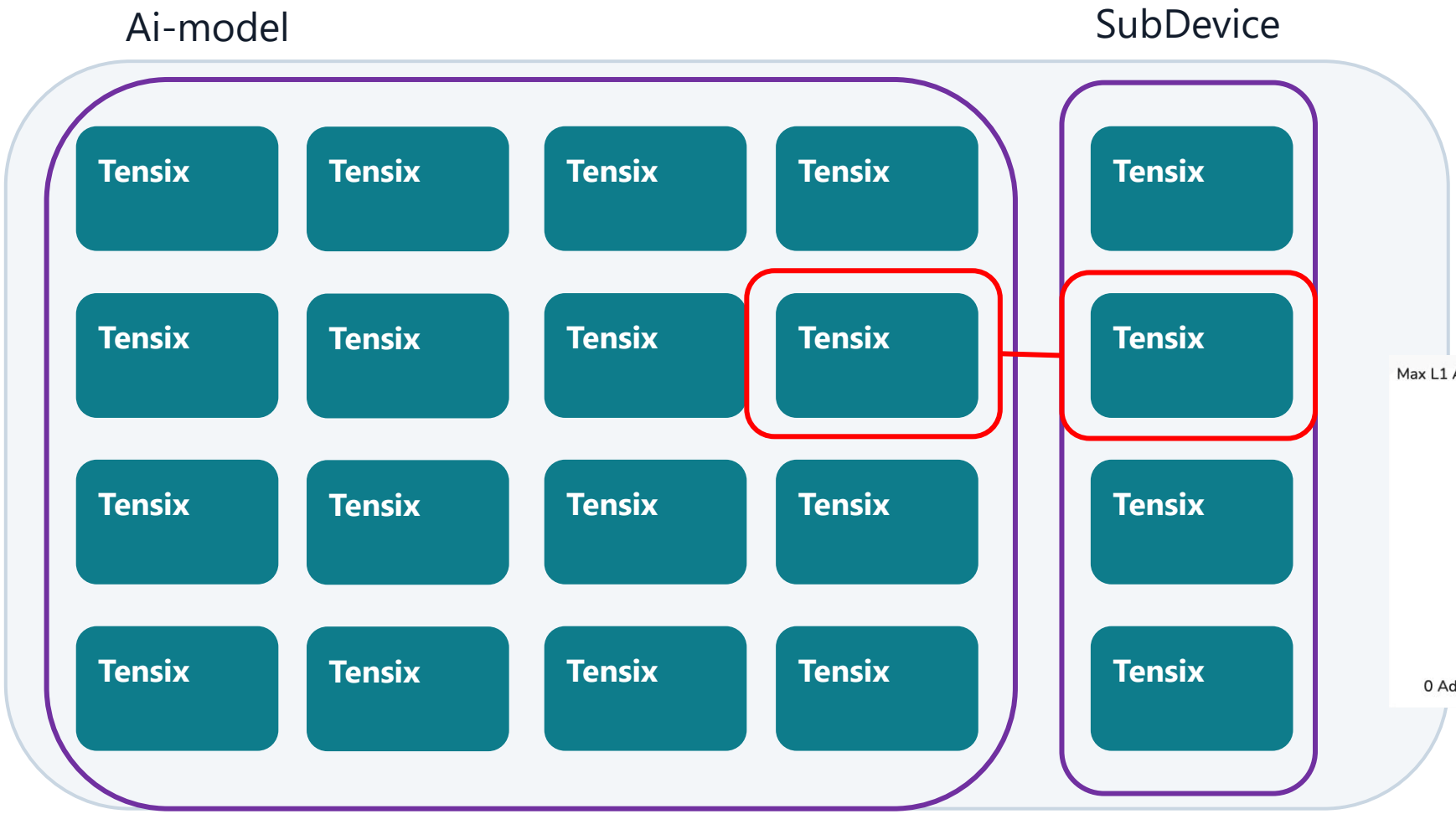
```
CORE_SET_14 = ttnn.CoreRangeSet(
    {
        ttnn.CoreRange(ttnn.CoreCoord(0, 0), ttnn.CoreCoord(4, 1)),
        ttnn.CoreRange(ttnn.CoreCoord(0, 2), ttnn.CoreCoord(3, 2)),
    }
)
CORE_SET_16 = ttnn.CoreRangeSet({ttnn.CoreRange(ttnn.CoreCoord(0, 0), ttnn.CoreCoord(3, 3))})
CORE_GRID_16 = ttnn.CoreGrid(x=4, y=4)
```

ResNet50의 경우 대부분의 Layer는 14~16 코어를 사용

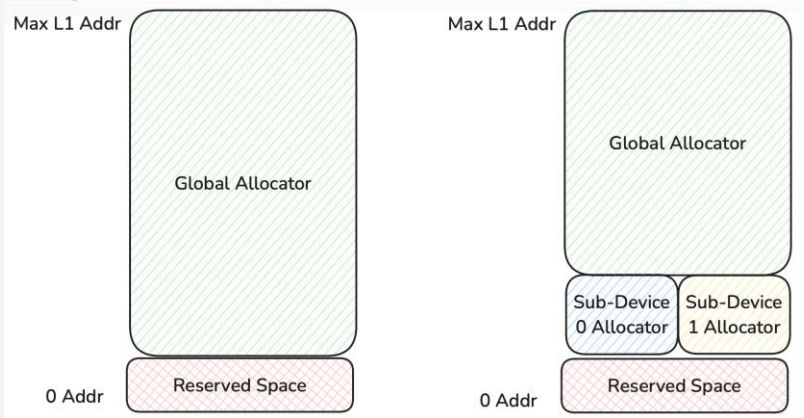




Sub-device Partition



- Global Semaphores
- Global Circular Buffers



https://github.com/tenstorrent/tt-metal/blob/main/tech_reports/SubDevices/SubDevices.md

Sub-device Partition

```
2026-07-14 05:07:39.180 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:_forward_layer:769 - ==== Running layer3 module4
2026-07-14 05:07:39.180 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:151 - ==== Running 1, 14, 14, 1024, 256
2026-07-14 05:07:39.180 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:158 - Running conv1
2026-07-14 05:07:39.188 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:238 - Running conv2
2026-07-14 05:07:39.203 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:302 - Running conv3
2026-07-14 05:07:39.210 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:_forward_layer:769 - ==== Running layer3 module5
2026-07-14 05:07:39.210 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:151 - ==== Running 1, 14, 14, 1024, 256
2026-07-14 05:07:39.210 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:158 - Running conv1
2026-07-14 05:07:39.218 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:238 - Running conv2
2026-07-14 05:07:39.232 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:302 - Running conv3
2026-07-14 05:07:39.239 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:_forward_layer:769 - ==== Running layer3 module6
2026-07-14 05:07:39.239 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:151 - ==== Running 1, 14, 14, 1024, 256
2026-07-14 05:07:39.240 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:158 - Running conv1
2026-07-14 05:07:39.247 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:238 - Running conv2
2026-07-14 05:07:39.260 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:302 - Running conv3
2026-07-14 05:07:39.268 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:_forward_layer:769 - ==== Running layer4 module1
2026-07-14 05:07:39.268 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:151 - ==== Running 1, 14, 14, 1024, 512
2026-07-14 05:07:39.268 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:158 - Running conv1
2026-07-14 05:07:40.139 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:238 - Running conv2
2026-07-14 05:07:41.679 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:302 - Running conv3
2026-07-14 05:07:42.592 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:run_downsample_if_req:384 - Running downsample
2026-07-14 05:07:44.712 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:_forward_layer:769 - ==== Running layer4 module2
2026-07-14 05:07:44.712 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:151 - ==== Running 1, 7, 7, 2048, 512
2026-07-14 05:07:44.712 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:158 - Running conv1
2026-07-14 05:07:45.597 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:238 - Running conv2
2026-07-14 05:07:47.094 | DEBUG    | models.bos_model.resnet50_cys.tt.ttnn_functional_resnet50:__call__:302 - Running conv3
```

TT-NN 레벨에서 ResNet50을 16개 코어로 제한.
FC layer 제외 구현 성공 -> TT-metalium 레벨에서 메모리 사용 최적화 필요.

향후 계획

- 1주차 · HW 아키텍쳐 학습
- 2주차 · SW 스택 이해
- 3주차 · AI model 포로파일링 및 연구주제 탐구
- 4주차 · 여유 코어 확보 및 SubDevice 탐구
- 5-6주차 · 연구 활동
- 7-8 · 연구 데이터 확보 및 최적화